

10 Academy: Artificial Intelligence Mastery

Week 8 Challenge Document

Date: 24 Jun – 30 Jun 2026

Shipping a Data Product: From Raw Telegram Data to an Analytical API

An end-to-end data pipeline for Telegram, leveraging dbt for transformation, Dagster for orchestration, and YOLOv8 for data enrichment.

Overview

Business Need

You are a Data Engineer at **Kara Solutions**, a leading data science consultancy in Ethiopia. Your team has been tasked with building a robust data platform that generates actionable insights about Ethiopian medical businesses, using data scraped from public Telegram channels.

A well-designed data platform significantly enhances data analysis. To achieve this, you will build an end-to-end pipeline that answers key business questions such as:

- What are the top 10 most frequently mentioned medical products or drugs across all channels?
- How does the price or availability of a specific product vary across different channels?
- Which channels have the most visual content (e.g., images of pills vs. creams)?
- What are the daily and weekly trends in posting volume for health-related topics?

Situational Overview

To answer these questions, you will implement a modern ELT (Extract, Load, Transform) framework. Raw data will be extracted from Telegram and loaded into a Data Lake storage zone. From there, it will be loaded into a PostgreSQL database serving as the data warehouse. The transformation step happens inside the warehouse using dbt, where you will clean the data and remodel it into a dimensional star schema optimized for analytical queries. This layered approach ensures data is reliable, scalable, and ready for analysis.

This project involves scraping, data modeling, object detection with YOLO to enrich the data, and exposing the final insights through an analytical API.

Your job is to build a data product that does the following:

- Develop a reproducible project environment and secure pipeline.
- Develop a data scraping and collection pipeline to populate a raw data lake.
- Design and implement a dimensional data model (star schema) in a PostgreSQL data warehouse.
- Develop a data cleaning and transformation pipeline using dbt.
- Enrich the data using object detection on images with YOLO.
- Expose the final, cleaned data through an analytical API using FastAPI.

Data

You will be building a **data warehouse** for this project. The data comes from public Telegram channels that sell medical and pharmaceutical products in Ethiopia.

Telegram Channels to Scrape:

- [CheMed Telegram Channel](#) — Medical products
- [Lobelia Cosmetics](#) — Cosmetics and health products
- [Tikvah Pharma](#) — Pharmaceuticals
- Additional channels from et.tgstat.com/medicine

Data Fields You Will Collect:

Field	Description
message_id	Unique identifier for each message
channel_name	Name of the Telegram channel
message_date	Timestamp of the message
message_text	Full text content (product names, prices, descriptions)
has_media	Whether the message contains media
image_path	Path to downloaded image (if applicable)
views	Number of views on the message
forwards	Number of times the message was forwarded

Learning Outcomes

Skills

- Telegram API data extraction using Telethon
- Data modeling: designing and implementing a star schema
- ELT pipeline development: building layered pipelines (Raw → Staging → Marts)
- Infrastructure as Code (IaC) and environment management using Docker and `requirements.txt`
- Data transformation at scale using dbt (Data Build Tool)
- Data enrichment using object detection (YOLO)
- Analytical API development with FastAPI
- Data pipeline orchestration with Dagster
- Testing and validation of data systems
- Managing credentials and secrets using environment variables

Knowledge

- Principles of modern ELT vs. ETL architectures
- Layered data architecture (Data Lake, Staging, Data Marts)
- Best practices in data cleaning, validation, and transformation
- Structuring data for efficient analytical queries (dimensional modeling)
- Integrating unstructured data (image detection results) into a structured warehouse
- Best practices for deploying and maintaining reproducible data pipelines

Behaviors

- Starting foundational tasks (scraping, modeling) early to avoid downstream blockers
- Maintaining detailed documentation in code comments and README files
- Using GitHub Issues and Projects for task tracking
- Communicating proactively when blocked (e.g., API rate limits, environment issues)
- Adhering to professional work ethics and time management practices

Communication

- Documenting data architecture and modeling decisions
- Reporting on project outcomes and technical challenges

Team

Tutors:

- Kerod
- Mahbubah
- Feven

Key Dates

- **Challenge Introduction** — 10:30 AM UTC, Wednesday 24 June 2026
- **Interim Submission** — 8:00 PM UTC, Sunday 28 June 2026
- **Final Submission** — 8:00 PM UTC, Tuesday 30 June 2026

Communication & Support

- **Slack channel:** #all-week8
- **Office hours:** Mon–Fri, 08:00–15:00 UTC

Deliverables and Tasks to be done

Project Structure

```
None
medical-telegram-warehouse/
├── .vscode/
│   └── settings.json
├── .github/
│   └── workflows/
│       └── unittests.yml
├── .env # Secrets (API keys, DB passwords) — DO NOT COMMIT
├── .gitignore
├── docker-compose.yml # Container orchestration
├── Dockerfile # Python environment
├── requirements.txt
├── README.md
├── data/
├── medical_warehouse/ # dbt project
│   ├── dbt_project.yml
│   ├── profiles.yml
│   ├── models/
│   │   ├── staging/
│   │   └── marts/
│   └── tests/
├── src/
├── api/
│   ├── __init__.py
│   ├── main.py # FastAPI application
│   ├── database.py # Database connection
│   └── schemas.py # Pydantic models
├── notebooks/
│   └── __init__.py
├── tests/
│   └── __init__.py
└── scripts/
```

Task 1 — Data Scraping and Collection (Extract & Load)

Objective: Build a data scraping pipeline that extracts messages and images from Telegram channels and stores them in a raw data lake.

Instructions:

1. Set Up Telegram API Access

- Register your application at my.telegram.org to get API credentials.
- Install and configure the Telethon library.

2. Telegram Scraping

- Utilize the Telegram API or write custom scripts to extract data from public Telegram channels relevant to Ethiopian medical businesses. Use the following channels:
 - [CheMed Telegram Channel](#)
 - [Lobelia Cosmetics](#)
 - [Tikvah Pharma](#)
 - Additional channels from et.tgstat.com/medicine
- For each message, extract:
 - message ID,
 - date,
 - text content,
 - view count,
 - forward count, and
 - media information (if present).

3. Download Images

- When a message contains a photo, download it.
- Store images using an organized folder structure:
`data/raw/images/{channel_name}/{message_id}.jpg`.

4. Populate the Data Lake

- Store raw scraped data as JSON files.
- Use a partitioned directory structure:
`data/raw/telegram_messages/YYYY-MM-DD/channel_name.json`.
- Preserve the original data structure from the API.

5. Implement Logging

- Log which channels and dates have been scraped.
- Capture any errors (e.g., rate limiting, network issues).
- Store logs in a `logs/` directory.

Deliverables:

- Working scraper script (`src/scraper.py`)
- Raw JSON files in the data lake structure
- Downloaded images organized by channel
- Log files showing scraping activity

Task 2 — Data Modeling and Transformation (Transform)

Objective: Transform raw, messy data into a clean, structured data warehouse using dbt and dimensional modeling.

Raw data is messy and untrustworthy. [DBT](#) helps us build a reliable "data factory" to transform this raw material into a clean, trusted data product. It allows us to:

- Transform chaos into clean, structured tables.
- Model the data into a standard star schema, creating a single source of truth for analytics.
- Test data to prevent bad records from reaching users.
- Modularize logic, making the system scalable and easy to maintain.

Use dbt to transform the raw data and build a structured data warehouse optimized for analysis.

Instructions:

1. Load Raw Data to PostgreSQL

- Write a Python script that:
 - reads JSON files from your data lake,
 - loads them into a raw schema in PostgreSQL, and
 - creates a table `raw.telegram_messages` with all scraped fields.

2. Initialize dbt Project

Shell

```
pip install dbt-postgres
dbt init medical_warehouse
```

- Configure `profiles.yml` to connect to your PostgreSQL database
- set up the project structure.

3. Create Staging Models

- Create staging models in `models/staging/` to clean and standardize raw data:
 - Cast data types appropriately (dates, integers, etc.)
 - Rename columns to consistent naming conventions
 - Remove or filter invalid records (empty messages, nulls)
 - Add calculated fields (`message_length`, `has_image` flag)
- You should have one staging model per raw source (e.g., `stg_telegram_messages.sql`).

4. **Design and Implement Star Schema** Create a dimensional model with the following tables in `models/marts/`:

Dimension Tables:

- `Dim_channels`:
 - channel key (surrogate),
 - channel name,
 - channel type (Pharmaceutical/Cosmetics/Medical),
 - first/last post date,
 - total posts,
 - avg views
- `Dim_dates`:
 - date key,
 - full date,
 - day of week,
 - day name,
 - week of year,
 - month,
 - month name,
 - quarter,
 - year,
 - is_weekend

Fact Table:

- `fct_messages`:
 - message ID,
 - channel key (FK),
 - date key (FK),
 - message text,
 - message length,
 - view count,
 - forward count,
 - has_image flag

5. **Implement dbt Tests**

- In `models/marts/schema.yml`, add
 - `unique` and `not_null` tests on primary keys and critical columns.
 - `relationships` tests on foreign keys.

6. **Create a Custom Data Test**

- Write at least one custom test in the `tests/` directory. For example:
 - `assert_no_future_messages.sql` — ensure no messages have future dates
 - `assert_positive_views.sql` — ensure view counts are non-negative

- These SQL queries must return 0 rows to pass and enforce a key business rule.

7. Generate Documentation

Shell

```
dbt docs generate
dbt docs serve
```

- Add descriptions to all models and columns in `schema.yml`.

Deliverables:

- Complete dbt project with staging and mart models
- All dbt tests passing (`dbt test` shows no failures)
- Generated dbt documentation
- A section in your report explaining your star schema design decisions

Task 3 — Data Enrichment with Object Detection (YOLO)

Objective: Use computer vision to analyze images and integrate the findings into your data warehouse.

Pre-trained object detection models like YOLO detect general objects (bottles, people, etc.), not specific products. Your goal is to add analytical value from image analysis.

Instructions:

1. Set Up YOLO Environment

Shell

```
pip install ultralytics
```

- Use the YOLOv8 nano model (`yolov8n.pt`) for efficiency on standard laptops.

2. Implement Object Detection Script

- Create `src/yolo_detect.py` that:
 - Scans for images downloaded in Task 1
 - Runs YOLOv8 detection on each image
 - Records detected objects with confidence scores
 - Saves results to a CSV file

3. Create Classification Scheme

- Based on detected objects, categorize each image:
 - `promotional` — contains person + product (someone showing/holding item)
 - `product_display` — contains bottle/container, no person
 - `lifestyle` — contains person, no product
 - `other` — neither detected

4. Integrate with Data Warehouse

- Create `models/marts/fct_image_detections.sql`:
 - Load YOLO results into PostgreSQL
 - Join with `fct_messages` on `message_id`
 - Include: `message_id`, `channel_key`, `date_key`, `detected_class`, `confidence_score`, `image_category`

5. Analyze Results

- In your report, answer:
 - Do "promotional" posts (with people) get more views than "product_display" posts?
 - Which channels use more visual content?
 - What are the limitations of using pre-trained models for domain-specific tasks?

Deliverables:

- Object detection script (`src/yolo_detect.py`)
- Detection results CSV file
- `fct_image_detections` dbt model integrated into the warehouse
- Analysis of image content patterns in your report

Task 4 — Build an Analytical API

Objective: Expose your data warehouse through a REST API that answers business questions.

Instructions:

1. Set Up FastAPI Project

Shell

```
pip install fastapi uvicorn
```

- Create the API structure in the `api/` directory.
- Set up a database connection using SQLAlchemy.

2. Implement Analytical Endpoints

Create endpoints that query your dbt marts:

- **Endpoint 1 — Top Products:** Returns the most frequently mentioned terms/products across all channels.

None

GET /api/reports/top-products?limit=10

- **Endpoint 2 — Channel Activity:** Returns posting activity and trends for a specific channel.

None

GET /api/channels/{channel_name}/activity

- **Endpoint 3 — Message Search:** Searches for messages containing a specific keyword.

None

GET /api/search/messages?query=paracetamol&limit=20

- **Endpoint 4 — Visual Content Stats:** Returns statistics about image usage across channels.

None

GET /api/reports/visual-content

3. Add Data Validation

- Use Pydantic models in [api/schemas.py](#) to define request/response structures.
- Add proper error handling and HTTP status codes.

4. Document the API

- FastAPI automatically generates OpenAPI documentation; access it at [/docs](#) when the server is running.
- Add descriptions to all endpoints and parameters.

Deliverables:

- Working FastAPI application ([api/main.py](#))
- At least 4 analytical endpoints
- Pydantic schemas for request/response validation
- Screenshots of API documentation and example responses

Task 5 — Pipeline Orchestration

Objective: Automate your entire pipeline using an orchestration tool.

A collection of scripts is not a production pipeline. To make your workflow robust, observable, and schedulable, you will use **Dagster** for its excellent local development experience.

Instructions:

1. Install Dagster

Shell

```
pip install dagster dagster-webserver
```

2. Define Pipeline as a Dagster Job

- Convert your pipeline into Dagster "ops" (operations):
 - `scrape_telegram_data` — run the scraper
 - `load_raw_to_postgres` — load JSON to database
 - `run_dbt_transformations` — execute dbt models
 - `run_yolo_enrichment` — run object detection

3. Create the Job Graph

- Define dependencies between ops.
- Ensure proper execution order.

4. Launch and Test

Shell

```
dagster dev -f pipeline.py
```

- Access the Dagster UI at <http://localhost:3000>.
- Run the pipeline manually
- monitor execution and logs.

5. Add Scheduling

- Configure the pipeline to run daily.
- Set up alerts for failures.

Deliverables:

- Dagster pipeline definition
- Screenshots of the Dagster UI showing successful runs

Due Dates

Interim Submission — Sunday 28 June 2026, 8:00 PM UTC

GitHub Repository containing:

- Completed Task 1 (working scraper, populated data lake) and Task 2 (dbt project with staging and mart models, passing tests)

Interim Report:

- Description of your data lake structure
- Diagram of your star schema
- Summary of data quality issues encountered and how you addressed them

Feedback: You may not receive detailed comments on your interim submission but will receive a grade.

Final Submission — Tuesday 30 June 2026, 8:00 PM UTC

GitHub Repository containing:

- All completed tasks with clean, well-documented code that passes tests

Final Report (blog post format, suitable for Medium):

- Visual diagram of your entire data pipeline
- Diagram of your star schema with explanations
- Technical choices and justifications for each task
- Screenshots of: dbt documentation, API endpoints working, Dagster UI
- Reflection on challenges, key learnings, and potential improvements

Feedback: You will receive comments/feedback in addition to a grade for your final submission.

Other Considerations

- **Documentation:** Maintain detailed documentation in code comments and README files.
- **Collaboration:** Use GitHub Issues and Projects for task tracking.
- **Communication:** Regular check-ins, Q&A sessions, and a supportive community atmosphere.
- **Flexibility:** Acknowledge potential challenges (API rate limits, etc.) and communicate proactively.
- **Professionalism:** Emphasize work ethics and professional behavior.
- **Time Management:** Start early — Tasks 1 and 2 are foundational and block all downstream work.

Tutorials Schedule

In the following, the colour **purple** indicates morning sessions, and **blue** indicates afternoon sessions.

Wednesday (24 June)

- Introduction to the Challenge (Kerod)
- Telegram Scraping with Telethon and Data Lake Design (Feven)

Thursday (25 June)

- End-to-End Data Transformation: Data Warehouse Modeling (Star Schema) and dbt (Mahbubah)
- Data Enrichment with Modern Computer Vision (Feven)

Friday (26 June)

- Building Analytical APIs with FastAPI and Logging (Mahbubah)
- Data Pipeline Orchestration with dagster (Feven)

Monday (29 June)

- Q&A (Mahbubah)

Tuesday (30 June)

- Q&A (Feven)

References

Web Scraping

- [A Practical Introduction to Web Scraping in Python — Real Python](#)
- [Beautiful Soup: Build a Web Scraper with Python — Real Python](#)
- [Python Web Scraping Tutorial — GeeksforGeeks](#)
- [Scrapy — Official Documentation](#)
- [Selenium — Official Documentation](#)
- [Telethon — Official Documentation](#)

dbt

- [Introduction — dbt Documentation](#)
- [Getting Started Tutorial — dbt Documentation](#)
- [Best Practices: How We Structure Our Projects — dbt Documentation](#)
- [dbt Tutorial — YouTube](#)
- [dbt Tutorial — Start Data Engineering](#)

YOLO

- [YOLOv8 Documentation — Ultralytics](#)
- [Detection Tasks — Ultralytics](#)

Data Modeling & Architecture

- [Dimensional Modeling Techniques — Kimball Group](#)
- [Star Schema vs Snowflake Schema — Gurug9](#)
- [How to Build an ETL Pipeline in Python — ProjectPro](#)
- [Weather Data Stack with dbt and Dagster — Airbyte](#)

FastAPI

- [FastAPI — Official Documentation](#)
- [First Steps Tutorial — FastAPI](#)
- [FastAPI Guide — Real Python](#)
- [Pydantic and FastAPI Integration — Medium](#)

Docker

- [Docker for Python Developers — Docker Documentation](#)
- [Docker Compose Getting Started — Docker Documentation](#)

Orchestration

- [Dagster — Official Site](#)
- [Getting Started — Dagster Documentation](#)
- [Weather Data Stack with dbt and Dagster — Airbyte](#)